EF Core・LocalDB 入門

商品データを LocalDB に保存し、Web 画面に一覧表示する

対応環境　Visual Studio 2022（17.8 以降）／.NET 8.0／EF Core 8.0.x／LocalDB

Version 1.2

学習内容

前教材では Controller 内で作成していた商品データを、SQL Server LocalDB の Products テーブルへ保存し、EF Core で取得する形へ変更する。DB の作成、商品データの手入力、Controller と View による一覧表示までを順に行う。

実習の前提

Visual Studio 2022 のバージョン 17.8 以降、.NET 8.0、前教材の MvcBasicSample を使用する。Razor 入門の Product、HelloController、Views/Hello/Index.cshtml を残したまま進める。新しいプロジェクトは作成しない。

【重要】実習のプロジェクト名は MvcBasicSample に統一する。コード中の名前空間も同じ名前を使う。

構成

ORM と LocalDB の基礎

STEP 1〜3　パッケージと Model と DbContext を準備

STEP 4〜5　接続文字列と Program.cs を設定

STEP 6〜7　Migration を作成して DB へ適用

STEP 8〜9　空のテーブルを確認して商品を手入力

STEP 10〜11　Controller と View を新規作成

STEP 12　/Products/Index で表示確認

到達確認と問題の切り分け

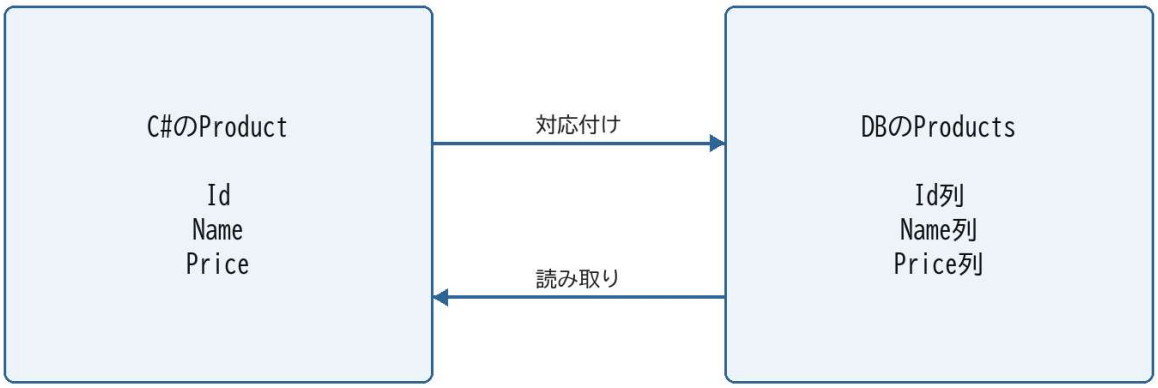
本版の内容

Version 1.2　STEP 1 の Installer 操作、LocalDB と SQL Server Data Tools の確認、NuGet の検索とバージョン選択、導入結果の確認を詳述

ORM と LocalDB の基礎

データベース（DB）はデータを保存して取り出す仕組みである。リレーショナルデータベースでは、表（テーブル）の行と列でデータを扱う。Products は商品を格納する表で、1 行が 1 商品、各列が Id、Name、Price に対応する。

ORM は Object-Relational Mapping の略で、C#のオブジェクトとデータベースの表を対応付ける仕組みである。Entity Framework Core（EF Core）は.NET 向けの ORM の一つである。



EF CoreがC#のオブジェクトと表のデータを対応付ける

1件のProduct ↔ Productsテーブルの1行

通常、表から値を取得するには SQL というデータベース操作用の言語を使う。EF Core は C#の問い合わせを SQL へ変換し、結果から Product などのオブジェクトを作る。SQL や表の基本知識が不要になるわけではない。

LocalDB は Windows 上で動く、開発向けの軽量な SQL Server である。EF Core は DB を操作する仕組み、LocalDB はデータを実際に保存する DB エンジンであり、役割が異なる。

【重要】クラスの定義、DB の構造、商品の実データは別のものである。この実習では Migration で構造を準備した後、商品を自分で手入力する。

STEP 1 実習環境とパッケージを準備する

対象ファイル

MvcBasicSample.csproj／Visual Studio の構成

新規作成か既存編集か

既存プロジェクトの設定変更。PC に不足する構成要素がある場合は、Visual Studio Installer で追加する。

操作手順

1 プロジェクトと Visual Studio のバージョンを確認する

① Visual Studio 2022 で、前回使用した MvcBasicSample.sln を開く。新しいプロジェクトは作成しない。ソリューション エクスプローラーに MvcBasicSample が表示されることを確認する。

② 実行中の場合は［デバッグ］→［デバッグの停止］（Shift＋F5）を選ぶ。［ファイル］→［すべて保存］で編集内容を保存する。

③ ［ヘルプ］→［Microsoft Visual Studio のバージョン情報］を開き、Visual Studio 2022 のバージョンが 17.8 以降であることを確認する。古い場合は、次の Installer で更新してから進める。

④ ソリューション エクスプローラーのプロジェクト MvcBasicSample を右クリックし、［プロパティ］を開く。［アプリケーション］→［全般］の［ターゲット フレームワーク］が.NET 8.0 になっていることを確認する。

［ソリューション エクスプローラー］が見当たらない場合は、［表示］→［ソリューション エクスプローラー］で開く。プロジェクトのプロパティ画面の配置は、Visual Studio の更新状況によって異なることがある。

2 Visual Studio Installer を開く

① Windows の［スタート］を開き、検索欄に Visual Studio Installer と入力して起動する。Installer は、Visual Studio の機能を追加・更新するための別のアプリである。

② ［インストール済み］に表示された Visual Studio 2022 の［変更］をクリックする。Community・Professional・Enterprise のうち、この実習で使っているものを選ぶ。

③ ［ワークロード］の［ASP.NET と Web 開発］が選択済みであることを確認する。ワークロードは、開発目的に合わせた機能のまとまりである。

Visual Studio の更新が必要な場合は、Installer の［更新］を先に実行する。更新後も使用する製品は Visual Studio 2022 である。

STEP 1 LocalDB とデータ操作ツールを確認する

操作手順 続き

3 LocalDB を確認する

① Installer の画面上部で［個別のコンポーネント］をクリックする。個別のコンポーネントは、機能を一つずつ選んで追加する画面である。

② 検索欄に LocalDB と入力する。［SQL Server Express 2019 LocalDB］を探し、チェックが付いていることを確認する。未選択の場合はチェックを付ける。

Visual Studio 2022 の構成一覧では、LocalDB の項目名に「2019」が含まれる。これは LocalDB の製品名であり、Visual Studio 2019 を使用するという意味ではない。

4 SQL Server Data Tools を確認する

① 検索欄の LocalDB を消し、SQL Server Data Tools と入力する。検索しても見つからない場合は、SQL と入力して一覧を確認する。

②［SQL Server Data Tools］にチェックが付いていることを確認する。未選択の場合はチェックを付ける。

SQL Server Data Tools は、Visual Studio からデータベースの構造や表の内容を確認するためのツールである。LocalDB はデータを保存する本体であり、役割が異なる。この実習では両方を用意する。

5 変更を適用して Visual Studio を開き直す

① チェックを追加した場合は、画面右下の［変更］をクリックする。Visual Studio を閉じるよう求められた場合は、保存済みであることを確認して閉じる。

② ダウンロードとインストールが完了するまで待つ。すでに両方が導入済みで、選択内容を変更していない場合は、追加インストールは不要である。

③ Visual Studio 2022 を起動し、MvcBasicSample.sln を開き直す。［表示］→［SQL Server オブジェクト エクスプローラー］を選び、ウィンドウが開くことを確認する。

この時点では Products テーブルが表示されなくてもよい。データベースとテーブルは、後の STEP で作成する。ウィンドウが開くことと、テーブルが存在することは別の確認である。

表示されない場合の確認

［SQL Server オブジェクト エクスプローラー］がメニューにない場合は、SQL Server Data Tools の導入を再確認し、Visual Studio を開き直す。［サーバー エクスプローラー］とは別のウィンドウなので、名称を確認する。

Installer で［変更］が使えない場合や、PC の管理者権限を求められた場合は、担当者に確認する。授業用 PC の構成を確認してから作業を進める。

STEP 1 NuGet パッケージを導入する

操作手順 続き

6 NuGet パッケージの管理画面を開く

① ソリューション エクスプローラーで、プロジェクト MvcBasicSample を右クリックし、［NuGet パッケージの管理］を選ぶ。一番上の「ソリューション」ではなく、その下のプロジェクトを選ぶ。

②［参照］タブをクリックする。画面上部の［パッケージ ソース］は nuget.org を選ぶ。［プレリリースを含める］のチェックは外す。

NuGet は、.NET の追加ライブラリをプロジェクトに取り込む仕組みである。Installer で行った PC 全体の機能追加とは異なり、ここでは MvcBasicSample が使用するライブラリを追加する。

7 SqlServer パッケージをインストールする

① 検索欄に Microsoft.EntityFrameworkCore.SqlServer と入力し、検索結果から同じ名前のパッケージを選ぶ。名前の末尾が SqlServer であり、作成者が Microsoft であることを確認する。

② 右側の詳細欄で［バージョン］の一覧を開き、8.0.31 を選ぶ。この実習では、2 つのパッケージを 8.0.31 に揃える。最初に表示される最新版をそのまま選ばない。

③［インストール］をクリックする。変更内容の確認画面が表示された場合は、MvcBasicSample への追加内容を確認して［OK］または［適用］を選ぶ。ライセンスの同意画面が表示された場合は内容を確認し、［同意する］を選ぶ。

④ インストールが完了するまで待つ。他の依存パッケージが一緒に追加されても、そのまま進めてよい。

8 Tools パッケージをインストールする

①［参照］タブの検索欄を Microsoft.EntityFrameworkCore.Tools に置き換える。検索結果から同じ名前で、作成者が Microsoft のパッケージを選ぶ。

②［バージョン］で 8.0.31 を選び、［インストール］をクリックする。確認や同意の画面が表示された場合は、先ほどと同様に進める。

9 2 つのバージョンを確認する

［インストール済み］タブを開き、SqlServer と Tools の両方が 8.0.31 になっていることを確認する。検索欄に文字が残っていると片方しか表示されないため、検索欄を空にして確認する。

8.0.x の「x」は修正版の番号を表す記号であり、文字 x を入力するわけではない。この実習では、具体的な番号として 8.0.31 を選択する。

STEP 1 導入結果を確認する

完成コード

この STEP では C#コードを入力しない。NuGet の操作によって、Visual Studio が MvcBasicSample.csproj にパッケージ参照を追加する。次の 2 行は、導入結果を確認するための抜粋である。既存の.csproj 全体をこの 2 行で置き換えない。

```
<PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.31" />
<PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.31" />
```

必要な場合は、プロジェクト MvcBasicSample を右クリックし、［プロジェクト ファイルの編集］で確認できる。上の参照は通常、ItemGroup 要素の内側に記録される。NuGet 画面で確認できれば、手入力による編集は不要である。

コードの意味

Microsoft.EntityFrameworkCore.SqlServer は、EF Core から SQL Server を操作するためのライブラリである。LocalDB も SQL Server の一種なので、このライブラリを使用する。

Microsoft.EntityFrameworkCore.Tools は、Visual Studio のパッケージ マネージャー コンソールで、Migration を作成したり DB へ適用したりするためのツールである。Migration は、Model から DB の構造変更を記録する仕組みである。

PackageReference の Include はパッケージ名、Version は使用するバージョンを示す。本書では.NET 8.0 と EF Core 8.0 系を組み合わせるため、SqlServer と Tools を同じ 8.0.31 に揃える。

実行確認

①［ビルド］→［ソリューションのビルド］を選ぶ。［出力］で成功を確認し、［エラー一覧］にビルドエラーがないことを確認する。

②［インストール済み］で SqlServer と Tools が両方 8.0.31、プロジェクトのターゲット フレームワークが.NET 8.0 であることを再確認する。

③［表示］→［SQL Server オブジェクト エクスプローラー］を開く。ここまで確認できたら、Model の作成へ進む。データベースへの接続確認は、DB を作成した後に行う。

うまく進まない場合

検索結果が出ない場合：パッケージ ソースが nuget.org であること、検索した名前が正しいこと、インターネットに接続できることを確認する。

インストール時に NU1202 が出た場合：パッケージと.NET の対応が合っていない可能性がある。選んだバージョンが 8.0.31 であり、プロジェクトが.NET 8.0 であることを確認する。

ビルドで失敗した場合：［エラー一覧］の最初のエラーを確認する。パッケージ導入前からあるコードのエラーと、パッケージの復元に関するエラーを分けて調べる。

STEP 2 Product を DB 用の Model にする

対象ファイル

Models/Product.cs

新規作成か既存編集か

既存編集

操作手順

1　Razor 入門で作成した Product.cs を開く。同名のクラスを追加せず、全体を下のコードに置き換える。

完成コード

```
using System.ComponentModel.DataAnnotations;

namespace MvcBasicSample.Models;

// Products テーブルの 1 行に対応する商品
public class Product
{
    public int Id { get; set; }     // 主キー

    // 商品名を必須の項目として扱う
    [Required]
    public string Name { get; set; } = string.Empty;

    public int Price { get; set; }  // 円単位の価格
}
```

コードの意味

Id は各商品を区別する主キーである。EF Core は Id という名前を規則により主キーとして認識する。SQL Server では、この int 型の主キーは通常、登録時に値を自動生成する列になる。

[Required]は Name を必須として扱う属性で、ここでは DB 列を NULL 不可にする意味も持つ。NULL は値が存在しない状態であり、空文字列とは異なる。

string.Empty は C#側の初期値であり、DB へ商品名を登録する処理ではない。Price は前教材と同じく円単位の整数を使う。

実行確認

ビルドしてエラー0 件を確認する。/Hello/Index は C#内で作った 2 件を引き続き表示できる。まだ DB とは接続していない。

【補足】[Required]だけで、SQL Server Object Explorer から入力する空文字列まで禁止されるわけではない。今回の手入力では商品名を空欄にせず、価格も正しく入力する。

STEP 3 AppDbContext を新規作成する

対象ファイル

Data/AppDbContext.cs

新規作成か既存編集か

Data フォルダとクラスを新規作成

操作手順

1　プロジェクトを右クリックし、［追加］→［新しいフォルダー］で Data を作る。

2　Data を右クリックし、［追加］→［クラス］で AppDbContext.cs を作り、内容をすべて下のコードに置き換える。

完成コード

```
using Microsoft.EntityFrameworkCore;  // EF Core を使用
using MvcBasicSample.Models;          // Product を使用

namespace MvcBasicSample.Data;

// EF Core を使ってデータベースへ接続するクラス
public class AppDbContext : DbContext
{
    // Program.cs で登録した接続設定を受け取る
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)               // 受け取った設定を親クラスへ渡す
    {
    }

    // Products テーブルを Product 型として問い合わせるためのプロパティ
    public DbSet<Product> Products => Set<Product>();
}
```

コードの意味

DbContext は EF Core を使って DB へ接続し、データを問い合わせたり変更したりするための基底クラスである。AppDbContext は DbContext を継承した、このアプリ専用のクラスである。

DbContextOptions<AppDbContext>には接続先や使用する DB の種類などの設定が入る。コンストラクターで受け取り、base(options)で DbContext へ渡す。設定内容は STEP 5 で Program.cs に登録する。

DbSet<Product>は、Products テーブルの各行を Product 型として問い合わせたり変更したりするためのプロパティである。Products というプロパティ名が、今回のテーブル名 Products になる。

=> Set<Product>()は、get で Set<Product>()の結果を返すプロパティの短い書き方である。ここだけで商品が読み込まれたり、テーブルが作られたりするわけではない。

実行確認

ビルドのエラー0 件を確認する。Data が Controllers や Models の中ではなく、プロジェクト直下にあることも確認する。

STEP 4 接続文字列を設定する

対象ファイル

appsettings.json／appsettings.Development.json

新規作成か既存編集か

2 ファイルとも既存編集

操作手順

1　プロジェクト直下の appsettings.json を開き、部分追記せず下の完成形へ全体を置き換える。

2　appsettings.Development.json も次の完成形へ置き換える。appsettings.json の下に折りたたまれている場合がある。

完成コード

appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=(localdb)¥¥MSSQLLocalDB;Database=MvcBasicSampleDb;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

【注意】DefaultConnection の値は紙面では折り返して見えるが、引用符で囲まれた 1 本の文字列である。途中へ Enter による改行を入れない。JSON 内に説明用コメントは書かない。

appsettings.Development.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

STEP 4 接続文字列の意味と確認

コードの意味

JSON は名前と値を対応させ、{ }で階層を作る形式である。項目間はカンマで区切り、最後の項目の後にはカンマを付けない。

```
appsettings.json
├── ConnectionStrings
│    └── DefaultConnection
├── Logging
│    └── LogLevel
└── AllowedHosts
```

【重要】ConnectionStrings と Logging と AllowedHosts は同じ階層に置く。ConnectionStrings を Logging の中へ入れると、通常の GetConnectionString では取得できない。

接続文字列の項目	意味
Server=(localdb)¥MSSQLLocalDB	接続する LocalDB のインスタンス
Database=MvcBasicSampleDb	今回使用する DB 名
Trusted_Connection=True	現在の Windows ユーザーで認証する
TrustServerCertificate=True	この開発接続で証明書検証を省略する

```
実際の接続先名　(localdb)¥MSSQLLocalDB
JSON の文字列内　(localdb)¥¥MSSQLLocalDB
```

JSON ではバックスラッシュ 1 文字を表すために 2 文字続けて書く。SQL Server Object Explorer のサーバー名では 1 文字、JSON の引用符内では 2 文字となる。Trusted_Connection の下線の前にバックスラッシュは不要である。

【補足】バックスラッシュは日本語の書体や環境では円記号（¥）に見えることがある。本書のコードでは同じ半角文字を表している。

appsettings.Development.json は開発環境で重ねて読み込まれ、同名の設定があれば優先される。本書では接続文字列を appsettings.json のみに置く。環境変数などの同名設定も上書きの要因になる。

実行確認

保存し、括弧、引用符、カンマのエラーがないことを確認する。次の STEP で起動でき、STEP 7 で MvcBasicSampleDb が作成されれば、接続設定を通した確認が完了する。

STEP 5　Program.cs に DB の設定を登録する

対象ファイル

プロジェクト直下の Program.cs

新規作成か既存編集か

既存編集

操作手順

1　Program.cs を下の完成形へ全体置換する。AddDbContext は builder.Build()より前に置く。

完成コード

```
using Microsoft.EntityFrameworkCore;  // UseSqlServer を使用
using MvcBasicSample.Data;            // AppDbContext を使用

var builder = WebApplication.CreateBuilder(args);

// Controller と View を使う MVC の機能を登録する
builder.Services.AddControllersWithViews();

// DefaultConnection という名前の接続文字列を取得する
var connectionString = builder.Configuration
    .GetConnectionString("DefaultConnection")
    ?? throw new InvalidOperationException("接続文字列がありません");

// AppDbContext を生成するときに使用する SQL Server の接続設定を登録する
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(connectionString));

var app = builder.Build();            // 設定からアプリを生成

// 開発環境以外で使うエラー画面と HTTPS の設定
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();            // HTTP から HTTPS へ転送
app.UseStaticFiles();                 // wwwroot にある CSS や JavaScript などを配信
app.UseRouting();                     // URL と Controller の Action を対応付ける
app.UseAuthorization();               // 設定されているアクセス許可を確認

// URL を Controller と Action に対応付ける
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();                            // Web サーバーを起動して要求の受信を開始
```

STEP 5　コードの意味と実行確認

コードの意味

GetConnectionString("DefaultConnection")は ConnectionStrings 内の DefaultConnection を読む。取得できなければ、?? throw により「接続文字列がありません」というエラーで停止する。??は左側が null のときに右側を使う演算子である。

AddDbContext は、ASP.NET Core が AppDbContext を生成するときに使用する設定を登録する。UseSqlServer は使用する DB を SQL Server に指定し、接続文字列を設定する。options => ...は、引数 options へ行う処理を表すラムダ式である。

登録した AppDbContext を、必要なクラスのコンストラクターへ ASP.NET Core が渡す仕組みを DI（Dependency Injection／依存性注入）という。Controller 側で new AppDbContext(...)と作成せず、Program.cs に登録した設定を使って生成された AppDbContext を受け取る。

Build 以降は MVC の起動処理である。前半だけを残して後半を消すとアプリを起動できず、Migration 用の設定取得にも失敗する。

実行確認

保存してビルドし、/Hello/Index が引き続き表示されることを確認する。この Action は DB を使わないので、DB 作成前でも動作する。

確認後は Shift＋F5 で停止する。起動成功だけでは DB 接続の確認にはならず、実際の DB 作成は STEP 7 で行う。

登録から受け取りまで

```
Program.cs で接続文字列を読む
    ↓
AddDbContext<AppDbContext>で生成方法を登録
    ↓
ASP.NET Core が必要な AppDbContext を用意
    ↓
ProductsController のコンストラクターで受け取る
```

【補足】AddDbContext の標準設定では、原則として 1 回の Web 要求を処理する間だけ同じ AppDbContext を使用する。アプリの起動中に 1 個の AppDbContext を使い続ける設定ではない。

STEP 6 Migration を作成する

対象ファイル

Migrations フォルダ内の自動生成ファイル

新規作成か既存編集か

新規作成。EF Core のツールが生成する

操作手順

1 ［ツール］→［NuGet パッケージマネージャー］→［パッケージマネージャーコンソール］を開く。通常の PowerShell やターミナルとは異なる。

2 コンソール上部の［既定のプロジェクト］を MvcBasicSample にする。プロジェクトを右クリックし、スタートアッププロジェクトにも設定する。

3 Migrations 内に InitialCreate がまだないことを確認し、次のコマンドを 1 回実行する。

完成コード

```
Add-Migration InitialCreate
```

入力はこのコマンドで完了する。生成ファイルはツールの版などで細部が変わるため手入力しない。生成された Up メソッドには Products の作成処理が入り、Id、Name、Price が定義される。

コードの意味

Migration は C#の Model をもとに DB 構造の変更内容を記録したものである。Add-Migration は変更手順のファイルを作り、この時点では DB へ適用しない。

Migrations には InitialCreate を含むファイルと、現在の構造を記録する ModelSnapshot が作られる。ファイル名の自動識別部分は手元の生成結果を使う。

実行確認

Build succeeded.だけで判断せず、その後に例外がなく処理が完了し、生成ファイルがあることを確認する。Up 内の Products と 3 列を確認する。

この時点で DB がまだ見えなくても問題ない。

【注意】成功済みの Add-Migration InitialCreate を繰り返さない。既に存在する場合は内容を確認して STEP 7 へ進む。コンソールに残る過去のエラーと今回の結果を区別する。

STEP 7　Migration を DB へ適用する

対象ファイル

MvcBasicSampleDb／Migrations の既存ファイル

新規作成か既存編集か

DB とテーブルを新規作成。作成済みなら未適用分を反映

操作手順

1　Migration 生成が完了していることを確認する。

2　同じパッケージマネージャーコンソールで次のコマンドを実行する。

完成コード

```
Update-Database
```

コードの意味

Update-Database は Migration を接続先 DB へ適用する。DB がなければ作成し、Products テーブルなどを作る。

__EFMigrationsHistory は適用済み Migration を記録する管理用テーブルで、商品を格納する表ではない。

実行確認

コマンドがエラーなく完了し、通常は Done.と表示されることを確認する。続く STEP 8 で実際の DB を確認する。

操作	作成または変更するもの
Add-Migration InitialCreate	プロジェクト内の DB 構造変更用ファイル
Update-Database	接続先 DB のテーブルと適用履歴

【重要】本書の Migration はテーブル構造を作るもので、商品データは登録しない。初回の DB 作成後は Products テーブルの中は空である。

【注意】実習済みの DB では過去のデータが残ることがある。空でないことを理由に既存 DB やテーブルを削除しない。DB 名と接続先を確認して実習済みの状態として扱う。

STEP 8 DB と空の Products テーブルを確認する

対象ファイル

編集するファイルなし。SQL Server Object Explorer の MvcBasicSampleDb を確認

新規作成か既存編集か

確認のみ。データはまだ追加しない

操作手順

1　［表示］→［SQL Server オブジェクト エクスプローラー］を開く。

2　SQL Server 配下の(localdb)¥MSSQLLocalDB を展開する。なければ［SQL Server の追加］でサーバー名に (localdb)¥MSSQLLocalDB を入力し、Windows 認証で接続する。

3　［データベース］を右クリックして［更新］する。MvcBasicSampleDb を展開し、［テーブル］も更新する。

4　dbo.Products と dbo.__EFMigrationsHistory を確認する。

5　dbo.Products を右クリックして［データの表示］を選ぶ。Id、Name、Price 列があり、登録済みの行が 0 件であることを確認する。

完成コード

この STEP ではコードを入力しない。SQL Server Object Explorer で、次の階層を順に展開して確認する。

```
(localdb)¥MSSQLLocalDB
└── データベース
    └── MvcBasicSampleDb
        └── テーブル
            ├── dbo.Products
            └── dbo.__EFMigrationsHistory
```

コードの意味

dbo はテーブルをまとめる名前（スキーマ）である。新規入力用の空行は、登録済みの商品には数えない。

実行確認

初回の実習では Products が 0 件なら正しい。この時点では商品一覧画面もまだ作成していない。

【注意】別の LocalDB インスタンスではなく MSSQLLocalDB の MvcBasicSampleDb を見る。DB が見えなければ、更新操作と接続文字列の Database を確認する。

STEP 9 サンプル商品を手入力する

対象ファイル

編集するファイルなし。SQL Server Object Explorer の dbo.Products を編集

新規作成か既存編集か

既存テーブルへデータ追加

操作手順

1　dbo.Products の［データの表示］を開く。

2　新規行の Name にコーヒー、Price に 500 を入力する。Id は自動採番されるため入力しない。

3　Tab などで次の行へ移動して変更を確定する。同様に紅茶 450、ジュース 300 を入力し、合計 3 件にする。

4　画面を更新するか閉じて再表示し、3 件が保存されたことを確認する。

完成コード

この STEP では C#や SQL のコードを入力しない。SQL Server Object Explorer の表へ、次の 3 件を直接入力する。

Id	Name	Price
自動採番	コーヒー	500
自動採番	紅茶	450
自動採番	ジュース	300

コードの意味

Id は商品を一意に識別する番号である。1、2、3 になることが多いが、過去の登録履歴によって別の値でもよい。

Price には数値だけを入力する。「円」はデータに含めず表示時に View で付ける。

実行確認

開き直しても 3 件が残り、Id が付いていれば成功。エラー表示があれば未確定なので、商品名、価格、入力中の行を確認する。

/Hello/Index にはまだ C#で作った商品が表示される。DB の 3 件を表示する画面は STEP 10 と 11 で作成する。

【重要】商品データを入れてから Controller と View を作成する。データの保存と画面表示を区別して確認する。

DB から読み取るコードの前提

次の STEP では ProductsController で DB の商品を取得する。コードに現れる用語を先に確認する。

DI で AppDbContext を受け取る

ProductsController(AppDbContext db)は ProductsController のコンストラクターである。Program.cs に登録した生成方法に従い、ASP.NET Core が AppDbContext を作成して引数 db へ渡す。受け取った値はフィールド _db へ保存し、Index()から使用する。readonly は、コンストラクターで設定した_db へ別の値を再代入できないようにする指定である。

一覧の問い合わせと非同期処理

LINQ は C#でデータの問い合わせを表す仕組みである。OrderBy(product => product.Id)は Id の昇順に並べる指定であり、ラムダ式で並び順の基準を選んでいる。

ToListAsync()は問い合わせを実行し、結果を一覧として取得する非同期メソッドである。非同期処理は、DB の応答を待つ間にスレッドという実行単位を占有し続けず、完了後に続きへ進める仕組みである。

記法	意味
Task<T>	後で T 型の結果が得られる処理を表す
async	メソッド内で await を使うための指定
await	完了を待ち、得られた結果を取り出す
Task<IActionResult>	後で画面などの応答結果を返す Action
ToListAsync()の結果	await 後に List<Product>として受け取る

【重要】return View(products)は一覧の取得完了後に実行される。非同期にしただけで DB の検索自体が速くなるわけではない。

```
ProductsController.Index()
    ↓ AppDbContext の Products へ問い合わせ
EF Core が SQL を実行して商品を取得
    ↓ await で結果を受け取る
List<Product>
    ↓ return View(products)
Views/Products/Index.cshtml
```

STEP 10 ProductsController を新規作成する

対象ファイル

Controllers/ProductsController.cs

新規作成か既存編集か

新規作成。HelloController は残す

操作手順

1　Controllers を右クリックし、［追加］→［クラス］で ProductsController.cs を作る。

2　内容をすべて下のコードに置き換える。名前は複数形の Products とする。

完成コード

```
using Microsoft.AspNetCore.Mvc;          // MVC の機能を使用
using Microsoft.EntityFrameworkCore;     // ToListAsync を使用
using MvcBasicSample.Data;               // AppDbContext を使用

namespace MvcBasicSample.Controllers;

// データベースの商品一覧を表示する Controller
public class ProductsController : Controller
{
    private readonly AppDbContext _db;  // DB へ問い合わせるためのフィールド

    // ASP.NET Core から必要な AppDbContext を受け取る
    public ProductsController(AppDbContext db)
    {
        _db = db;                       // 受け取った AppDbContext をフィールドに保存
    }

    // /Products/Index で商品一覧を取得する
    public async Task<IActionResult> Index()
    {
        // Id の昇順で取得し結果を List<Product>にする
        var products = await _db.Products
            .OrderBy(product => product.Id)
            .ToListAsync();

        return View(products);          // 商品一覧を View へ渡す
    }
}
```

コードの意味

_db.Products は Products テーブルに対する問い合わせを組み立てるための DbSet<Product>である。OrderBy で Id の昇順を指定し、ToListAsync で問い合わせを実行して全商品を取得する。

await 後の products は List<Product>型になる。return View(products)は、その商品一覧を次の STEP で作成する View へ渡す。

実行確認

ビルドのエラー0 件を確認する。View はまだないため、/Products/Index の表示確認は次の STEP の後に行う。

STEP 11 商品一覧の View を新規作成する

対象ファイル

Views/Products/Index.cshtml

新規作成か既存編集か

Products フォルダと View を新規作成

操作手順

1　Views を右クリックし、［追加］→［新しいフォルダー］で Products を作る。Hello の中には作らない。

2　Products 内に空の Razor ビューIndex.cshtml を作る。@page は付けず、全体を下のコードに置き換える。

完成コード

```
@model List<MvcBasicSample.Models.Product>
@{
    ViewData["Title"] = "商品一覧";  // タブ用のタイトル
}

<h1>商品一覧</h1>
@* 件数が 0 なら説明だけを表示する *@
@if (Model.Count == 0)
{
    <p>商品はありません</p>
}
else
{
    <p>@Model.Count 件の商品があります</p>
    <ul>
    @foreach (var product in Model)
    {
        <li>@product.Name @product.Price 円</li>
    }
    </ul>
}
```

コードの意味

@model List<MvcBasicSample.Models.Product>は商品 1 件ではなく一覧を受け取る宣言である。Controller が渡す List<Product>と一致させる。

Model.Count で件数を調べ、foreach で商品を 1 件ずつ表示する。テーブルが空なら「商品はありません」と表示する。

実行確認

保存してビルドする。続く STEP 12 で URL と DB データの表示をまとめて確認する。

STEP 12　商品一覧の表示を確認する

対象ファイル

編集するファイルなし。ProductsController.cs、Views/Products/Index.cshtml、dbo.Products を照合

新規作成か既存編集か

確認のみ

操作手順

1　https プロファイルで F5 を実行し、URL のパスを/Products/Index にする。

2　「商品一覧」「3 件の商品があります」と 3 件の商品が表示されることを確認する。

3　SQL Server Object Explorer でも同じ DB の Products を開き、商品名と価格を照合する。

完成コード

この STEP ではファイルを変更しない。ブラウザには次の内容が表示される。

```
商品一覧
3 件の商品があります

・コーヒー 500 円
・紅茶 450 円
・ジュース 300 円
```

コードの意味

/Products/Index は ProductsController の Index()を呼ぶ。DB の値を List<Product>へ読み込み、View へ渡して HTML にする。

/Hello/Index は C#内で作ったデータ、/Products/Index は DB のデータを表示する。同じ Index.cshtml でもフォルダが異なるため別の画面である。

実行確認

表示値が DB の値と一致すれば成功。停止して再起動しても DB へ保存した商品は残る。

Controller が Product1 件を渡すコードになっていないこと、View が List<Product>を宣言していることも確認する。

【補足】0 件なら完成 View は「商品はありません」と表示する。見出しだけや空の表示になる場合は、古い View を開いていないかと DB の接続先を確認する。

問題の切り分けと到達確認

症状	確認する内容
Add-Migration が認識されない	Tools の導入とパッケージマネージャーコンソールの使用
DbContextOptions を解決できない	AddDbContext が Build より前にあり、コンストラクターの型が一致しているか
接続文字列を取得できない	ConnectionStrings の階層とキー名と設定の上書き
DB が見えない	Update-Database の成功、接続先、表示の更新
View が見つからない	Views/Products/Index.cshtml の場所と名前
Model の型が合わない	Controller と View の両方が List<Product>か
商品が表示されない	DB 名、手入力の確定、開いている URL

到達確認

- ☐ ORM と DbContext と DbSet の役割を説明できる
- ☐ ConnectionStrings の階層と JSON のバックスラッシュを説明できる
- ☐ Add-Migration と Update-Database の違いを説明できる
- ☐ 空のテーブルを確認してから商品を手入力できる
- ☐ Product と List<Product>を区別して Controller と View の型を揃えられる
- ☐ DB の値が/Products/Index に表示される流れを説明できる

参照資料

Microsoft Learn　Migration とコンソールツール

https://learn.microsoft.com/ef/core/managing-schemas/migrations/

https://learn.microsoft.com/ef/core/cli/powershell

Microsoft Learn　SQL Server Express LocalDB

https://learn.microsoft.com/sql/database-engine/configure-windows/sql-server-express-localdb